

# Reviewer Pack — Minimal Entanglement Tensor Rank vs BP\_ReadTwice Circuit Thr...

Entry 65341f910458 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 22:19:22 UTC

## Minimal Entanglement Tensor Rank vs BP\_ReadTwice Circuit Threshold

Entry ID: 65341f910458 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 22:19:22 UTC

### 1. Conjecture

**Field A** (mathematical branch): Quantum Information Theory (Entanglement Tensor Networks)

**Field B** (complexity object): Complexity Theory (BP\_ReadTwice Circuit Complexity)

#### Statement:

{'st1': 'For any Boolean function  $f$  with  $n$  variables, the minimal rank of an entanglement tensor network  $T$  that certifies its computational hardness under BP\_ReadTwice is  $\Theta(\log(\Delta(f)))$ ', 'st2': 'where  $\Delta(f)$  denotes the maximum degree of any input variable in  $f$ .', 'st3': 'Additionally, for any entanglement tensor network  $T$  with rank  $O(n)$ , there exists a Boolean function  $f$  such that  $\Delta(f) = O(n^c)$  and  $\rho(T) = \Omega(n^{\{1+c\}})$  for some constant  $c > 0$ .']}

#### Rationale (proposer's reasoning):

{'st1': 'Entanglement tensor networks, which describe quantum correlations, have been shown to be useful in certifying the hardness of Boolean functions.', 'st2': 'The BP\_ReadTwice complexity captures the computational power of branching programs with two-way communication.', 'st3': 'A connection between entanglement and BP\_ReadTwice could provide a novel approach to proving lower bounds for circuit complexities.'}

**Taxonomy category:** BP\_READTWICE (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 5336b0b14e56f676

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if, for all generated Boolean functions  $f$  with  $n$  variables and  $\Delta(f)$ , the ratio of the rank of the entanglement tensor network  $T$  to  $\Delta(f)$  is within  $\Theta(\log(\Delta(f)))$  with at least 80% of seeds. It is falsified if any seed produces a ratio outside this range or if there exists an entanglement tensor network  $T$  with  $\rho(T) < \Omega(n^{\{1+c\}})$  for any constant  $c > 0$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | SAFE             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | UNCERTAIN | 1.00       | HITS             | UNCERTAIN        |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "entanglement tensor network" AND "BP\_ReadTwice circuit complexity" - "quantum information theory" AND minimal rank entanglement tensor" - "computational hardness" AND entanglement tensor network BP\_ReadTwice

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.5s

#### 5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n, max_degree):
        function = []
        for _ in range(2**n):
            inputs = [random.randint(0, 1) for _ in range(n)]
            output = random.choice([0, 1])
            if len(set(inputs)) <= max_degree:
                function.append((inputs, output))
        return function

    def construct_entanglement_tensor_network(function):
        # Simplified construction for demonstration
        rank = len(function)
        return rank

    def bp_readtwice_complexity(rank, n):
        # Simplified complexity calculation for demonstration
        return rank * n

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        max_degree = min(n, 10) # Limiting max degree to avoid trivial cases
        function = generate_boolean_function(n, max_degree)
```

```

rank = construct_entanglement_tensor_network(function)
complexity = bp_readtwice_complexity(rank, n)

if complexity == 0:
    continue

ratio = rank / max_degree
results.append({
    "n": n,
    "max_degree": max_degree,
    "rank": rank,
    "complexity": complexity,
    "ratio": ratio
})

if not results:
    return {
        "metric_name": "Rank vs BP_ReadTwice",
        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

mean_ratio = sum(result["ratio"] for result in results) / len(results)
std_ratio = math.sqrt(sum((result["ratio"] - mean_ratio)**2 for result in results) / len(results))

conjecture_holds = all(0.5 <= ratio <= 1.5 for result in results)
counterexample = "" if conjecture_holds else "Ratio outside [0.5, 1.5] range"

return {
    "metric_name": "Rank vs BP_ReadTwice",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    std_ratio = math.sqrt(sum((result["metric_value"] - mean_ratio)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])

```

```

print(f"RESULT: FALSIFIED counterexample=\"Ratio outside [0.5, 1.5] range\" first_failing_
else:
print(f"RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

TIMEOUT after 240s

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: Re-run the test with increased time limits to ensure it completes and produces results.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest      | 0         | 0   | 14174        |
| 2 | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6262         |
| 3 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4777         |
| 4 | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 5923         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13870        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 29822        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11476        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10816        |
| 9 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 45784        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142903 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in [research/audit/65341f910458.jsonl](#))

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/65341f910458.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/65341f910458.tar.gz` (if generated)